

Set Data Type

- We use set data type to store unique element because it will not accept duplicate element. (different)
- unordered data type, mutable data type. But element inside
- Set data type it will accept only immutable data type + single value data type.

Syntax:-

$V_n = \{e_1, e_2, e_3\}$

- If we sent mutable data type we get unhashable Type Error.
- We can't do indexing and slicing.
- In normal way we can't create set we can create set using object only.
Set()

default value of set data type is set()

$s = \{42, 42, 75, 65, 32\}$

$len(s)$

4

Methods :-

Adding methods.

1) add()

- It will add element randomly at any position.
- It will not take duplicate value.
- Add the element & return no output.

Syntax :- vn.add (element)

- element : single value data + immutable data type (we pass)
- if we pass (list, set, dict) mutable D.T. it shows unhashable error. Type error.
- can add only single-single element.
- If we want to add multiple element we can use () parenthesis.

b.add(("abc", 'b')) No error add directly.

b.add("abc", 'b')

Type Error

- Add with boundary condition.

2) update()

we can use nested list

[201307] ✓

[20130740] x

- It remove boundary condition while adding and return blank we we call variable then shows ^{updated a/p.}
- we can add multiple element at a time

x.update("abc", [111, 123], {66, 55})

- We use multivalue data string, list, tuple, set, dictary.

Syntax :- vn.update(iterable)

a = {20, 30}

a.update('python')

{'p', 20, 30, 't', 'n', 'o', 'y', 'h'}

([[[]]) - Error

Nested type No error (([]))

Page No.	
Date	

- It will won't accept nested list it shows unhashable error. Type error

a = [[["B"]]]

Type Error

a.update([["hello"]])

No error (tuple allow)

a.update([["hello"]])

Type Error

Remaining Method :-

pop()

It will remove random element (any element) and return the output as pop remove element. we can't predict exact output.

Syntax :- vn.pop()

we can't provide any argument in pop method. if we pass argument we get

Type error

remove()

It is used to remove specified element from set. and return output as blank space

Syntax :- vn.remove(element)

If that element is not present we get

Key Error

x = {1, 2, 3}

x.remove(2)

blank space

x

{1, 3}

5) discard()

It acts same as remove but if the element is not present it don't show any error it shows blank space.

Syntax: `vn.discard(element)`

`x = {1, 2, 3}`

`x.discard(2)`

`x`

`{1, 3}`

`x.discard(5)`

6) clear()

When we want to clear all elements from set we used clear and return no %P.

Syntax: `vn.clear()`

del keyword

When we want to remove memory location of that set.

`del vn`

7) intersection()

We use intersection to get common element in both set and if there is no any common element then it print `set()` as %P.

Syntax: `set1.intersection(set2)` reference
`vn1.intersection(vn2)` specified set

$x = \{10, 20, 30, 40, 50\}$
 $b = \{30, 40, 50, 'y', 'n'\}$

$x.intersection(y)$

$\{30, 40, 50\}$

$x = \{10, 20\}$

$y = \{40\}$

$x.intersection(y)$
 $set()$

we can use multiple set

$x.intersection(a, b, c)$

Symmetric-difference()

When we want to find or access uncommon element from both sets then we use Symmetric-difference.

If there is no any uncommon element then it print $set()$

Syntax: $Var1.Symmetric-difference(Var2)$

$x.Symmetric-difference(b)$

$\{10, 20, 'y', 'n'\}$

only one arg pass

difference()

When we want to find that what element are present in first set [original] but not in second set [reference]

Syntax: $Var1.difference(Var2)$

can pass multiple arg

$x.difference(y)$

$\{10, 20\}$

$b.difference(x)$

$\{'y', 'n'\}$

If no difference, we get $set()$ as o/p.

Boolean method.10) .isdisjoint()

When there is no (any) common element in both set then it print result as True otherwise false.

Syntax : $Vn_1.isdisjoint(Vn_2)$

eg:-

$x = \{4\}$

$y = \{5\}$

$x.isdisjoint(y)$

True

$x = \{4, 2\}$

$y = \{2\}$

$x.isdisjoint(y)$

False

11) .issuperset()

When all elements of set2 (reference set) are present in set1 (original set) then it shows True otherwise False.

Syntax : $Vn_1.issuperset(Vn_2)$

Original set

Reference set /
Specified set

eg: $x = \{4, 5\}$

$y = \{4\}$

$x.issuperset(y)$

True.

$y.issuperset(x)$

False.

12) issubset()

When all element of original set (set1) are present in reference set (set2) then it shows True otherwise false.

Syntax:- $Vn1.issubset(Vn2)$
 $set1.issubset(set2)$

True

$x = \{4, 5\}$

$y = \{5\}$

$x.issubset(y)$

False

$y.issubset(x)$

True

13) union()

When we want to merge two sets.

Syntax:- $Vn1.union(Vn2)$

pass multiple argument

$x = \{4, 5\}$

$y = \{5\}$

$x.union(y)$

$\{4, 5\}$

It create its own set not effect on original & specified set

14) copy()

Copy is the process of copying one variable into another. Both have same id.

Syntax:- $Vn1 = Vn2$
 $new Vn1 = Original Vn.$

15) intersection_update() :-

It returns a set that contains similarity between two or more sets (same value)

It will effect on original set (all element from specified set gets add to original set because of immutability)

original value changes and specified value will not change.

Syntax :- Set1.intersection_update(set2)

$a = \{42, 51, 36\}$

$b = \{36, 42, 19\}$

$b.intersection_update(a)$ multiple set can pass at time

$\begin{matrix} b \\ \{36, 42\} \end{matrix}$ only common element

$\begin{matrix} a \\ \{42, 51, 36\} \end{matrix}$

16) Symmetric - difference - update() :-

It is used to find ~~one~~ different value
It affect on original set.

original value changes and specified value will not change.

Syntax :- Set1.symmetric_difference_update(set2)

eg : $a = \{44, 22, 55\}$

$b = \{22, 11, 33\}$

$a.symmetric_difference_update(b)$

$\begin{matrix} a \\ \{33, 11, 44, 55\} \end{matrix}$

$\begin{matrix} b \\ \{33, 11, 22\} \end{matrix}$

17) difference-update()

It returns set that contains difference between two or more sets.

It will effect on original set
Original value changes & specified values will not change.

Syntax :- $\text{Set1} \cdot \text{difference-update}(\text{set2})$

$$a = \{15, 19, 51, 35, 15\}$$

$$b = \{15, 25, 65, 175\}$$

$$a \cdot \text{difference-update}(b)$$

$$\{35, 51\}$$

$$\{15, 25, 65, 175\}$$